Introducción a Python (III)

GIARA

Departamento de Automática y Computación

UPNA

Índice

- Lectura/Escritura de ficheros
- Definición de funciones
  - Ámbito de las funciones
  - Funciones lambda
  - Aplicación de funciones sobre estructuras de datos

Lectura/Escritura de ficheros

- El tratamiento de ficheros está implementado sobre el paquete stdio de C
- Abrir un fichero
  - file = open(nombre_archivo[, modo[, buffer]])
  - modo
    - ‘r’: read-only (por defecto). Es necesario que exista
    - ‘w’: write-only.
    - ‘a’: write-only. El contenido se añade al final (si el archivo existe)
    - ‘r+’: El archivo debe existir y se abre en lectura y escritura
    - ‘w+’: Lo mismo que r+ pero no es necesario que exista

Lectura/Escritura de ficheros

- Algunas funciones de un archivo f
  - f.close()
  - f.closed() -> True or false
  - f.mode
  - f.name
  - f.read()
    - Devuelve el contenido del fichero
  - f.readline()
    - Devuelve una línea del fichero

Lectura/Escritura de ficheros

- Cuando queremos leer todas las líneas de un fichero

```
fid = open('archivo','r')
for line in fid:
        instrucciones
```

- Similar a

```
fid = open('archivo','r')
line = fid.readline();
while line!="":
        line=fid.readline()
        instrucciones
```

Lectura/Escritura de ficheros

- Algunas funciones de un archivo f
  - f.seek(pos, how)
    - coloca la posición actual pos bytes más alla de how. Si how=0, significa desde el comienzo del archivo
- Escritura en fichero
  - fid.write('Línea a escribir')

```
fid = open('profesores.txt', 'r')
for lineas in fid:
    print "Linea de fichero: "+líneas
fid.close()
```

```
Linea de fichero: Nombre Apellido1 Apellido2 correo

Linea de fichero: Humberto Bustince Sola bustince@unavarra.es

Linea de fichero: Edurne Barrenechea Tartas
edurne.barrenechea@unavarra.es

Linea de fichero: Daniel Paternain Dallo daniel.paternain@unavarra.es

Linea de fichero: Aránzazu Jurío Munárriz aranzazu.jurio@unavarra.es
```

- Lo mismo que

```
fid = open(‘profesores.txt’, ‘r’)
línea = fid.read()
while línea!=“”:
        print "Linea de fichero: "+línea
        línea = fid.read()
```

Definición de funciones

- Función: objeto de Python con un nombre que hace que una serie de instrucciones se ejecuten bajo petición (llamada a la función)
- Instrucción def

```
def nombre_funcion(parámetros):
    instrucción o conjunto de instrucciones
```

Definición de funciones

- La lista de parámetros puede ser vacía o un conjunto de parámetros separados por coma
- Cuando llamamos a la función, pasamos valores a los parámetros -> argumentos (o bien no pasamos nada si no hay parámetros)
- Cuando ejecutamos el código correspondiente, la función no se ejecuta, sino que se ejecuta solo en las llamadas
- El cuerpo de la función puede contener cero o más instrucciones

  return variable

Definición de funciones

- Ejemplo

```
def cuadrado(x):
    return x**2
```

Definición de funciones

- Parámetros
  - los parámetros de entrada que se especifican con un literal son parámetros obligatorios

```
def cuadrado(x):
    return x**2
```

cuadrado(4) -> 16

cuadrado() -> Error

cuadrado(4,5) -> Error

cuadrado([4,5]) -> Error

Definición de funciones

- Parámetros
  - Podemos añadir después de los parámetros obligatorios parámetros de carácter optativo.
  - La forma de definirlos es mediante

    identificador = valor_por_defecto

  - Ejemplo

```
def cuadradoymas(x, y=0):
        return (x+y)**2


cuadradoymas(2) -> 4

cuadradoymas(2,1) -> 9
```

Definición de funciones

- Parámetros optativo de longitud variable
  - def función(param1, param2, *param)
    - *param: especifica una lista de valores que se agruparán en una tupla llamada param

```
def mifuncion(x, *lista):
        if lista is ():
                return x**2
        else:
                lis = [x**i for i in lista]
                return lis
```

mifuncion(5) -> 25

mifuncion(5, 2, 3, 4) -> [5, 25, 125]

Definición de funciones: ámbito de las variables

- Variables locales en la función
  - Las variables definidas dentro de una función son locales a la función

```
In [4]: def mifuncion():
            x = 5
            print x


        mifuncion()
        print x
```

```
5

---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-4-f945c889d796> in <module>()
      5
      6 mifuncion()
----> 7 print x

NameError: name 'x' is not defined
```

Definición de funciones: ámbito de las variables

- Variables locales en la función
  - Al revés es justo lo contrario. Las variables definidas fuera de las funciones actúan de manera global

```
In [15]: def mifuncion():
             print x


         x = 2
         mifuncion()
         print x
```

```
2
2
```

Definición de funciones: ámbito de las variables

- Variables locales en la función
  - Excepto si las variables se llaman de la misma forma. Entonces la variable x (dentro de la función) se utiliza únicamente en la función
    - ...Si el dato es inmutable...

```
In [6]: def mifuncion():
            x = 5
            print x


        x = 3
        mifuncion()
        print x
```

```
5
3
```

Definición de funciones: ámbito de las variables

- Variables locales en la función
  - Si queremos utilizar una variable global dentro de la función (y no ha sido pasada por parámetros) podemos especificarlo

```
In [19]: def mifuncion():
             global x
             x = 5
             print x


         x = 3
         mifuncion()
         print x
```

```
5
5
```

Definición de funciones: ámbito de las variables

- Variables locales en la función
  - Excepto si las variables se llaman de la misma forma. Entonces la variable x (dentro de la función) se utiliza únicamente en la función
    - ¿Y si son mutables?

```
In [10]: def mifuncion():
             x.append(4)
             print x


         x = [1,2,3]
         mifuncion()
         print x
```

```
[1, 2, 3, 4]
[1, 2, 3, 4]
```

Definición de funciones: ámbito de las variables

- OJO: Algún comportamiento inesperado

In [17]:
```
def mifuncion(x, y = []):
    x = x**2
    y.append(x)
    print y

mifuncion(3, [1, 4])
mifuncion(3)
mifuncion(5)
```

```
[1, 4, 9]
[9]
[9, 25]
```

Definición de funciones

- Devolución de múltiples resultados
  - Lo mejor es utilizar el formato tuple

```
def cuadradocubo(x):
    return x**2, x**3
```

```
x, y = cuadradocubo(5)
```

x -> 25

y -> 125

Definición de funciones: módulos

- Definición de módulos
  - Podemos definir un conjunto de funciones e instrucciones en un archivo .py
    - En este caso podemos hace referencia al conjunto de funciones/instrucciones como un módulo
    - Definimos un archivo potencias.py

```
def cuadrado(x):
        return x**2

def cubo(x):
        return x**3

def otraspotencias(x,y):
        return x**y
```

Definición de funciones: módulos

- Definición de módulos
  - En nuestro nuevo programa incorporamos la línea
    - import potencias
  - Esto nos permite hacer uso de las funciones cuadrado, cubo y otraspotenicas sin necesiad de tenerlas definidas en nuestro nuevo archivo .py
    - Simplemente haciendo llamadas del estilo

    x = potencias.cuadrado(5)

    y = potencias.cubo(6)

    z = potencias.otraspotencias(5,4)

Definición de funciones: módulos

- Definición de módulos
  - También podemos incorporar el nombre de una función concreta sin necesidad de especificar el nombre del módulo
    - from potencias import otraspotencias
  - En nuestro código podemos hacer referencia a

    z = otraspotencias(5,4)
  - Podemos traer un mayor número de funciones
    - from potencias import cuadrado,cubo
    - from potencias import * -> no es muy recomendable

Funciones lambda

- Un caso especial de función anónima (no tiene nombre) que se produce en tiempo de ejecución
  - Útiles cuando se utilizan junto a otras funciones como map/filter/reduce
  - son expresiones de una única instrucción
  - Se definen cuando se van a utilizar en un entorno muy concreto y no van a ser reutilizadas

```
lambda lista_parámetros : expresión

lambda x : x**2

lambda x,y,z: x+y+z
```

Funciones lambda

- Pueden ser asociadas a variables
  - f = lambda x,y x**2+y**2
  - f(5,6) -> 61
- Muy útiles cuando se utilizan con otras funciones
  - función max(arg1, arg2, key=función)
  - Queremos calcular el máximo en función de su último elemento. Por ejemplo : máximo de [4,5,1] y [2,2] es [2,2] porque el último elemento del segundo vecto es mayor

Funciones lambda

- Podría hacer

```
def ultimo(lista):
    return ultimo[len(lista)-1]

max([4, 5, 1], [2, 2], key=ultimo)
```

- O bien utilizando funciones lambda

```
max([4, 5, 1], [2, 2],key= lambda lista: lista[len(lista)-1])
```

Funciones sobre estructuras iterables

- Funciones para aplicar a elementos iterables
  - map(nombre_funcion, iterable)
    - Aplica la función a cada elemento del iterable

```
def cuadrado(x):
    return x**2

map(cuadrado, [1, 2, 3]) -> [1, 4, 9]


map(lambda x: x**2, [1, 2, 3]) -> [1, 4, 9]
```

Funciones sobre estructuras iterables

- Funciones para aplicar a elementos iterables
  - reduce(nombre_función, iterable [, inicial])
    - Aplica una función de dos argumentos a todos los elementos del iterable de la forma

función(elem1, elem2)

función(función(elem1,elem2), elem3)

...

función(función(...(función(elem1,elem2)))), elemn)

Funciones sobre estructuras iterables

- Funciones para aplicar a elementos iterables
  - reduce(nombre_función, iterable [, inicial])

```
def suma(x,y):
    return x+y
```

reduce(suma, [1, 5, 7, 2]) -> 15

-> suma(suma(suma(1,5),7),2)

reduce(lambda x,y: x+y, [1, 5, 7, 2]) -> 15

reduce(labda x,y: x+y, [1, 5, 7, 2], 10) -> 25

Si se añade un valor inicial, éste es cogido como primer elemento

Funciones sobre estructuras iterables

- Funciones para aplicar a elementos iterables
  - filter(nombre_función, iterable)
    - Devuelve un nuevo iterable con aquellos elementos de iterable para los que la función devuelve True

```
def positivo(x):
    return x>=0

filter(positivo, [-4, 5, 1, -2, -3, -2]) -> [5, 1]


filter(lambda x: x>=0, [-4, 5, 1, -2, -3, -2]) -> [5, 1]
```

Funciones sobre estructuras iterables

- Aplicación de map/filter sobre tuplas
  - La función map devuelve una lista
    - map(lambda x: x+1, (1,2,3)) -> [2, 3, 4]
  - La función reduce devuelve una tupla
    - map(lambda x: x>0, (-1, 2, -3)) -> (2,)
- Aplicación de map/filter/reduce sobre diccionarios
  - La función map actúa sobre los índices
    - map(lambda x: x+1, {'a': 0, 'b': 1}) -> Error
    - map(lambda x: len(x), {'a': 0, 'b': 1}) -> [1, 1]

Funciones sobre estructuras iterables

- Funciones para aplicar a elementos iterables
  - Este tipo de funciones pueden extenderse a otras funciones que ya existen
    - max(iterable [, key=función])
    - max(elem1, elem2 [, key=función])
    - min(iterable [, key=función])
    - min(elem1, elem2 [, key=función])
    - sorted(iterable [, key = función])